Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 8

The Same Calls, Inside Your Own OS

Track B: Build Your Own Small OS (xv6)

Done by: st. gr. FAF-241, Ermicev Sofia

Checked by: Patricia Reițman

Chișinău – 2026

Contents

1 Purpose of the Work **2**

2 Environment and Preparation **2**

3 Part 1. run inside xv6 **2**

3.1 Observe answers 3

4 Part 2. pingpong **3**

4.1 Observe answers 3

5 Comparison with Lab 7 **3**

6 Conclusion **4**

1. Purpose of the Work

The purpose of this laboratory work is to repeat the Linux process-control model inside xv6 and implement bidirectional parent-child communication using pipes. The work focuses on `fork()`, `exec()`, `wait()`, `pipe()`, `read()`, `write()`, and correct closing of unused descriptors.

2. Environment and Preparation

The new programs were added to xv6's `UPROGS` list:

```
$U/_run\
$U/_pingpong\
```

Figure 1.1 – Actual UPROGS entries used for Lab 8

3. Part 1. run inside xv6

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

Figure 2.1 – Actual run output when the child execs echo

The program was then changed temporarily to execute `ls`. The child printed the xv6 root directory and the parent reported successful termination:

```
parent: my pid is 3
child: my pid is 4
parent: child 4 ended with status 0
```

Figure 2.2 – Actual beginning/end of the successful ls exec experiment

Finally, an invalid program name was tested:

```
$ run
parent: my pid is 3
child: my pid is 4
```

```
exec failed
parent: child 4 ended with status 1
```

Figure 2.3 – Actual failed exec experiment in xv6

3.1. Observe answers

1. xv6 uses `fork()`, `exec()` and `wait()`; Linux Lab 7 used `fork()`, `execlp()` and `waitpid()`. The overall process model is the same.
2. The line `hello from the child` is printed by the `echo` program after successful `exec()`, not by the old `run` code.
3. The xv6 failure status is 1 because the failure path calls `exit(1)`. Lab 7 used `exit(127)`, so Linux reported 127 there.

4. Part 2. pingpong

```
$ pingpong
4: received ping
3: received pong
```

Figure 3.1 – Actual pingpong output in xv6

4.1. Observe answers

1. Two pipes are used because one carries data from parent to child and the other carries the reply from child to parent.
2. The parent waits for the reply in `read(c2p[0], &b, 1)`. The blocking `read()` cannot complete until the child writes one byte to `c2p[1]`.

5. Comparison with Lab 7

The same concepts appear in both laboratories. Lab 7 used Linux's richer interfaces such as `execlp()` and `waitpid()`, while Lab 8 used xv6's simpler `exec()` and `wait()`. Lab 7 built a one-direction pipeline between `ls` and `wc`; Lab 8 used two pipes so the parent and child could exchange data in both directions.

6. Conclusion

The experiments demonstrated that the fundamental process model is the same in xv6 and Linux: fork creates a child, exec replaces the current program, and wait collects child termination. The ping-pong exercise added explicit inter-process communication and showed how blocking reads naturally synchronize two processes.